Software Design Document (SDD)

Module	TIM Driver
Version	1.0.0
Team	Carlos Araiza · Carlos Cayetano · Kheara Kieley · Vania Leal
Date	26/04/2026
Institution	CETYS Universidad

---

1. Introduction

1.1. Purpose

This document describes the design and architecture of the TIM (Timer) driver module developed for the STM32F411RE microcontroller. Grounded in the requirements from the corresponding SRS, it covers data structures, hardware register interactions, control flow, and the complete public API. The document serves as the technical reference for implementing and validating any higher-level module that depends on timer functionality including blocking delay generation and PWM signal output.

1.2. Scope

The TIM driver is a bare-metal firmware component for the STM32F411RE platform. It abstracts the STM32F4 timer peripheral registers and exposes a validated API through which higher-level modules can enable timer clocks, configure time intervals or frequencies, start and stop timers, poll for update events, and set up capture/compare channels for PWM. Supported peripherals are TIM1 through TIM5 and TIM9 through TIM11. The clock assumption is 16 MHz HSI (default after reset, no PLL). All hardware access is performed through the CMSIS TIM_TypeDef and RCC_TypeDef structures no STM32 HAL is used.

1.3. Definitions and Acronyms

Term	Definition

TIM	Timer peripheral — a hardware counter in the STM32F4 that increments at a configurable clock rate and can generate events when it reaches a programmed threshold.
PSC	Prescaler Register — divides the input clock before it drives the counter, reducing the effective tick frequency.
ARR	Auto-Reload Register — the counter upper bound; when CNT reaches ARR an update event fires and CNT resets to 0.
CNT	Counter Register — the current value of the timer's free-running counter.
CR1	Control Register 1 — contains the CEN bit (counter enable) and the ARPE bit (auto-reload preload enable).
SR	Status Register — holds the UIF bit, which is asserted when an update event occurs.
UIF	Update Interrupt Flag — bit 0 of SR; set when CNT overflows to 0. Must be cleared by software.
CCR	Capture/Compare Register — the threshold compared against CNT to trigger output compare events or PWM transitions.
CCMR	Capture/Compare Mode Register — configures the operating mode of each channel (frozen, active, PWM 1, PWM 2, etc.).
CCER	Capture/Compare Enable Register — the CCxE bits enable each channel's output.
CEN	Counter Enable — bit 0 of CR1; setting it starts the timer counter.
ARPE	Auto-Reload Preload Enable — bit 7 of CR1; buffers ARR writes so they take effect at the next update event.
OCxPE	Output Compare Preload Enable — a bit in CCMRx that buffers CCR writes, preventing glitches on the PWM output.
PWM	Pulse Width Modulation — encodes a signal as the ratio of HIGH time to total period (duty cycle). Implemented in modes TIM_COMPARE_MODE_PWM1 and TIM_COMPARE_MODE_PWM2.
APB1 / APB2	Advanced Peripheral Bus 1 and 2. TIM2–TIM5 are clocked from APB1; TIM1, TIM9–TIM11 are clocked from APB2.
HSI	High Speed Internal oscillator — the default 16 MHz clock source on the STM32F411RE after reset.
CMSIS	Cortex Microcontroller Software Interface Standard — vendor-supplied register-map headers used for all hardware access in this driver.
SRS	Software Requirements Specification — the document defining the behavioral requirements this design implements.
SDD	Software Design Document — this document.
FR	Functional Requirement — a requirement specifying a concrete system behavior.

NFR	Non-Functional Requirement — a quality attribute requirement such as portability or determinism.

1.4. Requirements

All functional requirements specific to this module are defined in the corresponding SRS for the TIM driver. The table below traces each SRS requirement to the public API function that satisfies it.

Req. ID	Function	Requirement Statement
FR-1	tim_init()	The driver shall disable all supported TIM peripheral clocks as the initial reset state.
FR-2	tim_initTimer()	The driver shall enable the peripheral clock for any individually specified timer and reset its registers.
FR-3	tim_setTimerMs()	The driver shall configure PSC and ARR to generate an update event every the specified number of milliseconds.
FR-4	tim_setTimerFreq()	The driver shall configure PSC and ARR to generate update events at the specified frequency in Hz.
FR-5	tim_enableTimer()	The driver shall start a specified timer by asserting the CEN bit in CR1.
FR-6	tim_disableTimer()	The driver shall stop a specified timer by clearing the CEN bit in CR1.
FR-7	tim_waitTimer()	The driver shall block execution until the UIF flag is set, then clear it.
FR-8	tim_setTimerCompareChannelValue()	The driver shall write a compare value to the CCR register of the specified channel.
FR-9	tim_setTimerCompareMode()	The driver shall configure the output compare mode of a channel via CCMRx.
FR-10	tim_enableTimerCompareChannel()	The driver shall enable the output of a capture/compare channel via CCER.
FR-11	tim_disableTimerCompareChannel()	The driver shall disable the output of a capture/compare channel via CCER.

The SRS additionally defines NFR-1 (no external library dependencies other than stdint.h and stddef.h), NFR-2 (structured error handling on all public functions), and NFR-3 (deterministic execution for real-time embedded contexts). Clock assumptions are based on the STM32F411xC/E reference manual RM0383.

2. System Architecture

2.1. High-Level Design

The TIM driver follows the same layered architecture established by the GPIO driver. Each layer calls only into the layer immediately below it:

Layer	Description
Application Layer	main.c integrates all modules. It calls timer_delay_ms() for LED blinking and pwm_init/pwm_setSignal/pwm_start for PWM output. No application code accesses timer registers directly.
Timer Module Layer	timer.h / timer.c — wraps the TIM driver for blocking delay generation using TIM2. Exposes timer_init and timer_delay_ms.
PWM Module Layer	pwm.h / pwm.c — wraps the TIM driver (TIM3) and the GPIO driver (PA6, AF2) for PWM generation. Exposes pwm_init, pwm_setSignal, pwm_start, and pwm_stop.
TIM Driver Layer	tim_driver.h / tim_driver.c — the only layer that writes to timer hardware registers. Validates all inputs through is_valid_tim() and is_valid_channel(), enables clocks via RCC APB1ENR/APB2ENR, and performs direct operations on CR1, PSC, ARR, CNT, SR, CCRx, CCMRx, and CCER.
Hardware Layer	STM32F411RE TIM peripheral registers at their APB1/APB2 fixed memory addresses, accessed through the CMSIS TIM_TypeDef typed pointer macros (TIM1 … TIM5, TIM9 … TIM11).

2.2. Module Interfaces

The TIM driver exposes its entire functionality through the single public header file tim_driver.h. The module's interface characteristics are as follows:

- **Public API:** All callable functions are declared in tim_driver.h. These are tim_init, tim_initTimer, tim_setTimerMs, tim_setTimerFreq, tim_enableTimer, tim_disableTimer, tim_waitTimer, tim_setTimerCompareChannelValue, tim_setTimerCompareMode, tim_enableTimerCompareChannel, and tim_disableTimerCompareChannel.
- **Internal implementation:** All function bodies and the two private helpers (is_valid_tim and is_valid_channel) are defined in tim_driver.c and are not accessible to external modules.

- **Dependencies:** The driver depends on the CMSIS STM32F4xx device header (stm32f4xx.h) for the TIM_TypeDef and RCC_TypeDef structures, and on stddef.h for NULL. No HAL or external libraries are used; all hardware access is through direct register writes.
- **Hardware dependencies:** STM32F411RE TIM peripheral registers at their APB1 and APB2 fixed base addresses, and the RCC APB1ENR and APB2ENR registers for clock gating.

3. Detailed Design

3.1. Class Diagram

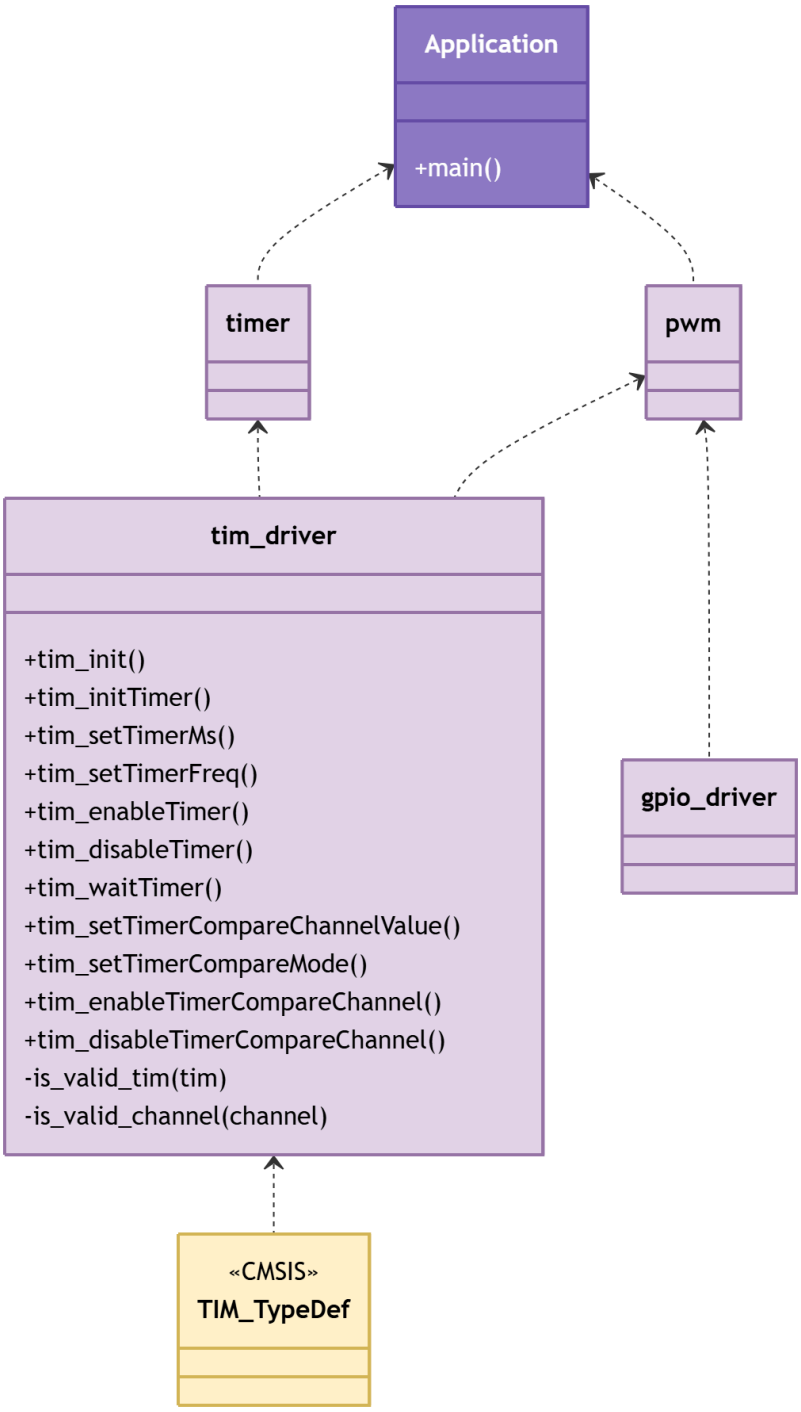

Figure 1. Class Diagram - Driver

3.2. Module Behavior (Sequence Diagrams, State Machines, etc.)

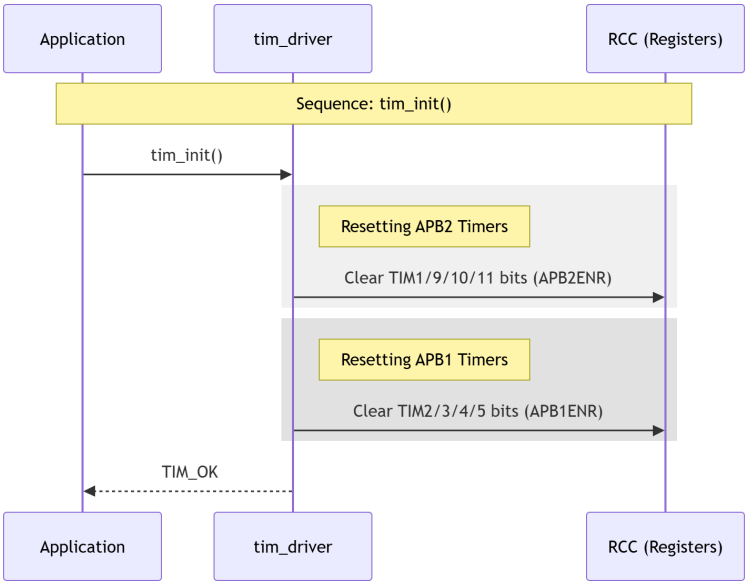

Figure 2. Sequence Diagram - Driver Init

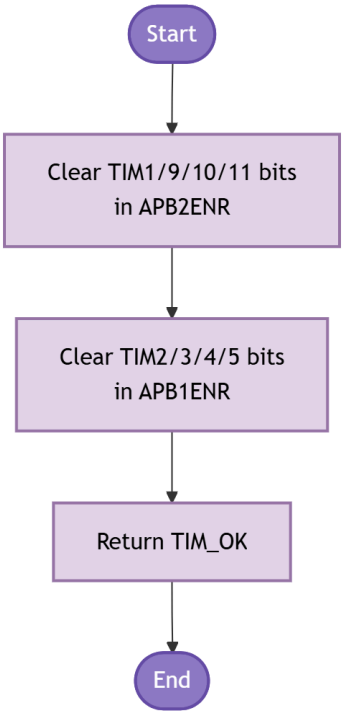

Figure 3. Activity Diagram - Driver Init

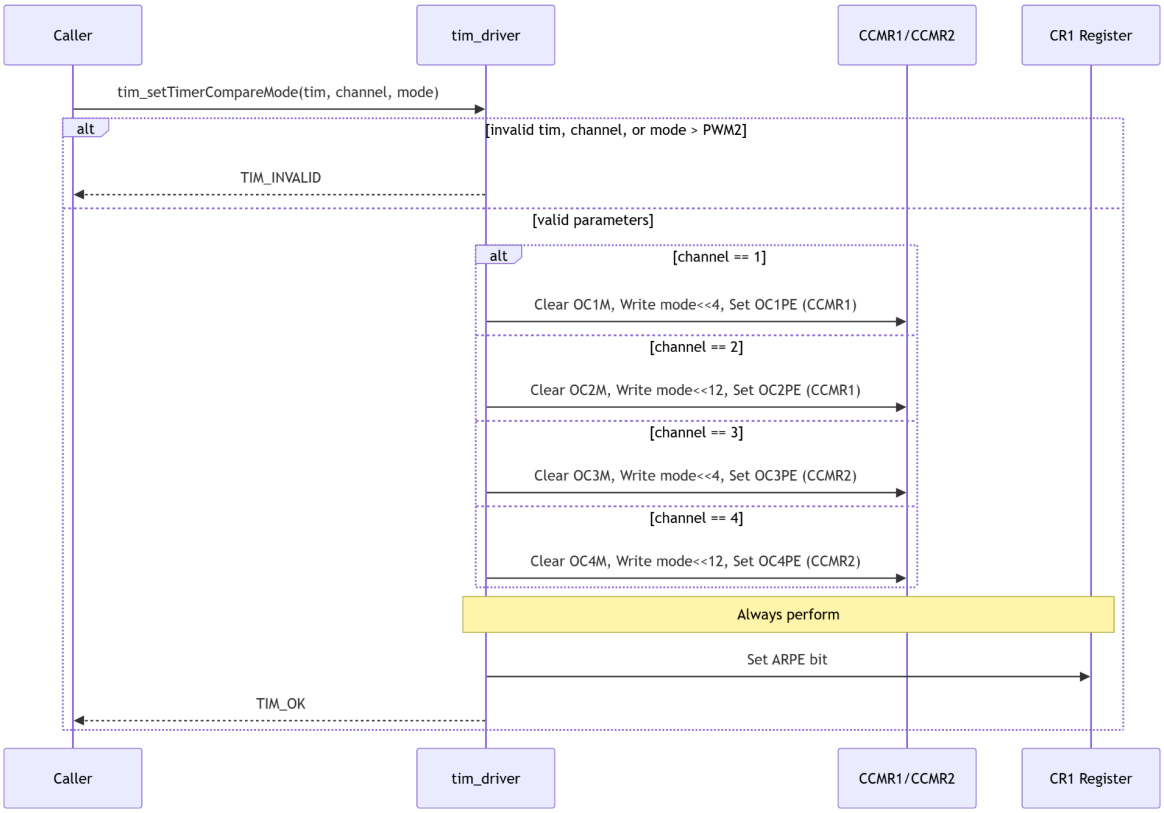

Figure 4. Sequence Diagram - Driver Action with Conditional

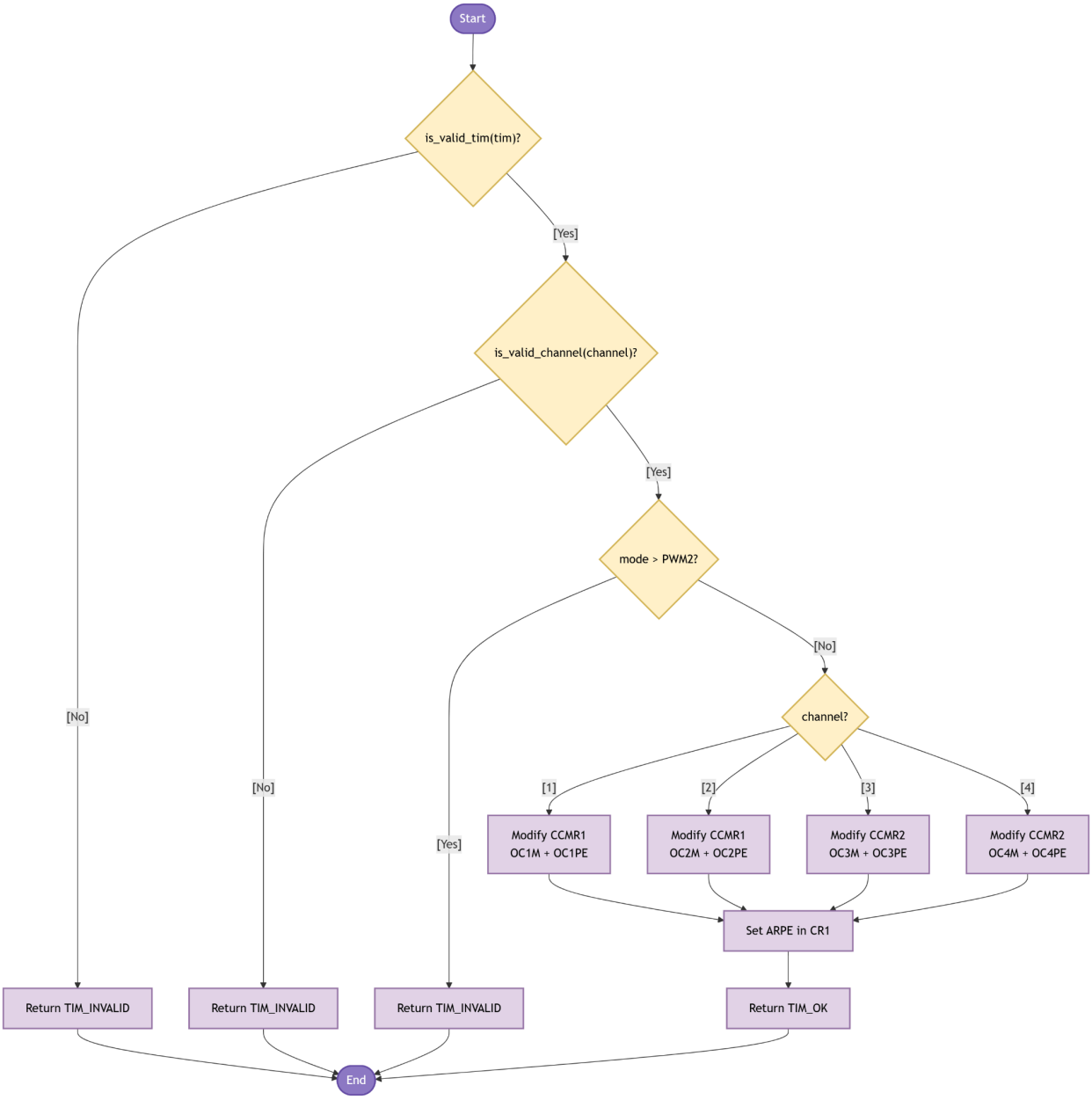

Figure 5. Activity Diagram - Driver Action with Conditional

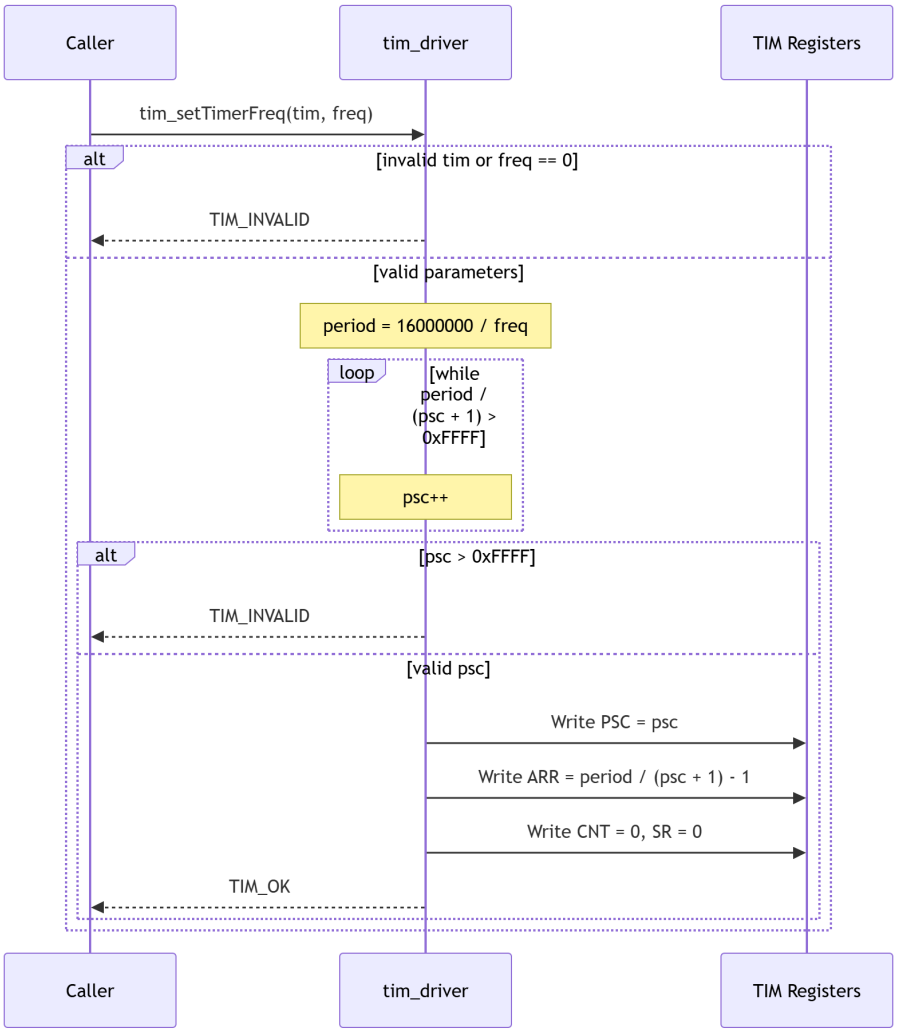

Figure 6. Sequence Diagram - Driver Action with Loop

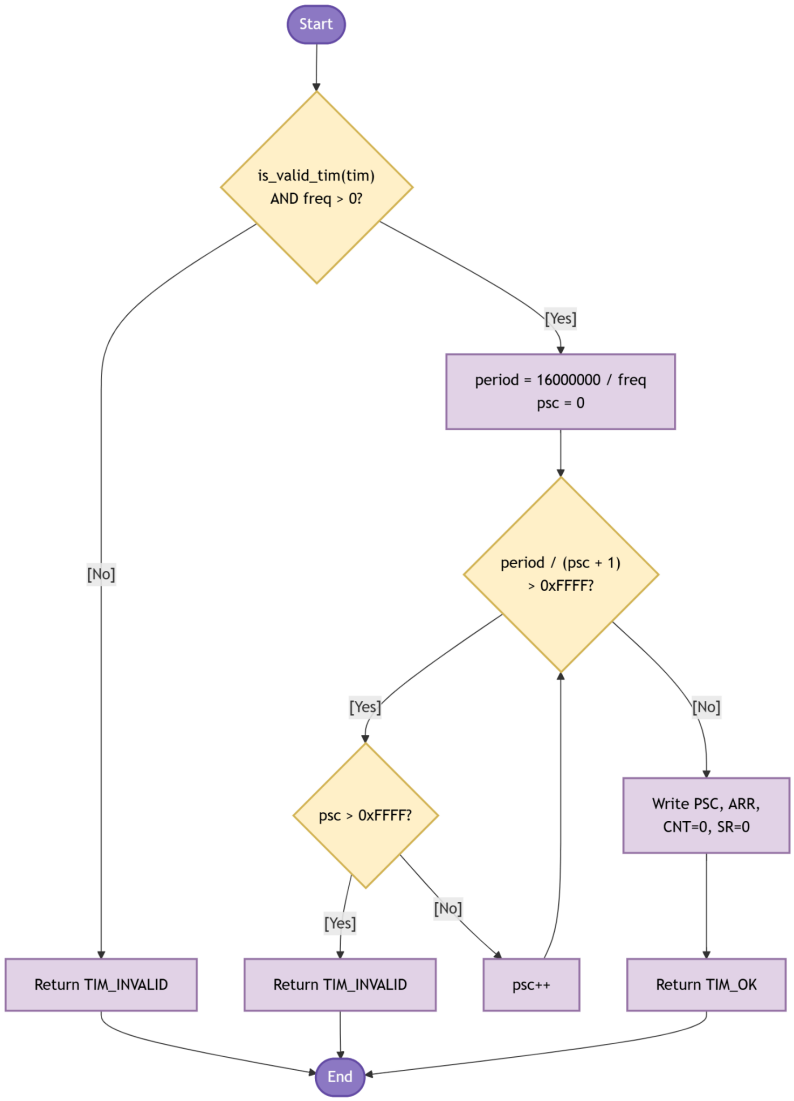

Figure 7. Activity Diagram - Driver Action with Loop

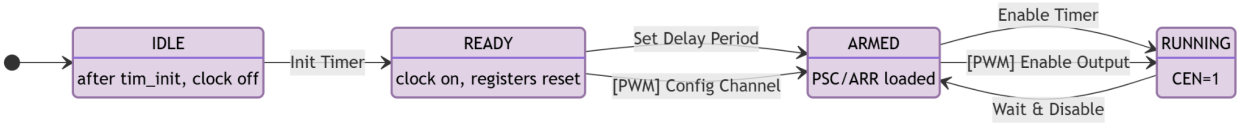

Figure 8. State Diagram - Driver

3.3. TIM Driver

The driver works in three phases: subsystem initialization, time-base configuration, and output-compare setup. The sections below walk through each function group, explain what registers are touched and why, and reference the diagrams above.

Subsystem initialization

Call tim_init() once at the start of main(), before any other TIM function. It clears the clock-enable bits for all eight supported timers in two register writes: one AND-NOT to APB2ENR (TIM1/TIM9/TIM10/TIM11) and one to APB1ENR (TIM2/TIM3/TIM4/TIM5). This leaves all timer clocks off and their registers inaccessible. Figures 2 and 3 show the sequence and activity.

After tim_init(), call tim_initTimer() for each timer you need. It enables the clock for that peripheral by OR-ing the matching APB1ENR or APB2ENR bit, then resets the time-base registers to a known state: CR1 = 0 (counter stopped), PSC = 0, ARR = 0xFFFF, CNT = 0, SR = 0. In this lab, timer.c calls tim_initTimer(TIM2) and pwm.c calls tim_initTimer(TIM3).

Time-base configuration for delays

tim_setTimerMs() programs the timer to generate one update event after a given number of milliseconds. It validates both arguments: the pointer via is_valid_tim() and the ms value (must be in [1, 65535]). Then it writes PSC = (16000000 / 1000) - 1 = 15999, giving a 1 kHz tick. ARR = ms - 1, so the counter overflows after exactly ms ticks. Both CNT and SR are zeroed before the counter is enabled.

timer.c wraps this into timer_delay_ms(ms): calls tim_setTimerMs(), then tim_enableTimer(), then tim_waitTimer(), then tim_disableTimer(). All four return codes are checked; the first error exits early.

Time-base configuration for PWM frequency

tim_setTimerFreq() programs the timer to fire update events at a target frequency. It validates tim and checks that freq is not zero. Then it computes period = 16000000 / freq. Because ARR is 16-bit (max 65535), a loop increments a local psc starting at 0 until period / (psc + 1) fits in 16 bits. If psc reaches 65535 without satisfying the condition, the function returns TIM_INVALID. Otherwise it writes PSC, ARR, and also resets CNT and SR to zero. Figure 6 and Figure 7 show the loop.

Counter enable, disable, and polling

tim_enableTimer() sets CEN (bit 0) in CR1 via an OR. tim_disableTimer() clears it via AND-NOT. The counter value in CNT is preserved on disable.

tim_waitTimer() polls SR bit 0 (UIF) in a loop until the counter overflows ARR. Then it clears the flag by writing SR &= ~TIM_SR_UIF. If tim_enableTimer() was never called, this loop will never exit. No interrupt vector is needed; the trade-off is that the CPU cannot do other work during the wait.

Output-compare channel configuration

tim_setTimerCompareMode() configures how a channel behaves when CNT reaches CCRx. Before writing anything, it checks all three arguments: is_valid_tim(), is_valid_channel(), and mode <= TIM_COMPARE_MODE_PWM2. Then, for each channel, it clears the OCxM field in CCMR1 or CCMR2 using the CMSIS mask constants, ORs in the new mode shifted to the right bit position, and sets OCxPE (output compare preload enable). Figures 4 and 5 show the channel conditional. After all channel branches, ARPE in CR1 is set unconditionally. Both preload flags (OCxPE and ARPE) are required: without them, a CCR or ARR write can take effect mid-cycle and produce a glitch on the output.

tim_setTimerCompareChannelValue() writes a 32-bit value into CCR1, CCR2, CCR3, or CCR4 using a switch on the channel. In PWM Mode 1, the output is HIGH while CNT < CCRx. Duty cycle = CCRx / (ARR + 1). pwm.c caches ARR after tim_setTimerFreq() returns and computes CCRx as (pwm_arr + 1) * duty_pct / 100.

Channel output enable and disable

tim_enableTimerCompareChannel() sets bit (channel - 1) * 4 in CCER. This connects the compare output to the physical GPIO pin, which must already be in alternate function mode (pwm.c calls gpio_setAlternateFunction(GPIOA, GPIO_PIN_6, 2) before this). tim_disableTimerCompareChannel() clears the same bit. The pin then follows the GPIO output data register.

In this lab, pwm_start() calls tim_enableTimerCompareChannel() then tim_enableTimer(). pwm_stop() calls tim_disableTimerCompareChannel() then tim_disableTimer(). The sequence matters: always enable the channel before starting the counter to avoid a brief spurious output.

4. Application Programming Interfaces

4.1. Data Types and Macros

tim_status_t: Return type for all TIM driver public functions. Every function validates its inputs and returns one of these codes before any register access.

tim_channel_t: Identifies a capture/compare channel (1–4). Used by all compare-channel functions.

tim_compare_mode_t: Selects the output compare operating mode written to the OCxM field in CCMRx. Enumerator values match the hardware 3-bit encoding directly.

TIM_SYSTEM_CLOCK_HZ: Macro set to 16000000UL. Represents the 16 MHz HSI clock frequency assumed by the prescaler calculations in tim_setTimerMs and tim_setTimerFreq.

tim_status_t: Enumerates the possible outcomes of a driver call.

Enumerator	Value	Meaning
TIM_OK	0	The operation completed without error.
TIM_ERROR	1	Generic error — reserved for hardware-level conditions not covered by TIM_INVALID.
TIM_INVALID	2	A parameter failed validation: timer pointer unrecognized, ms out of range, freq is 0, channel outside 1–4, mode exceeds TIM_COMPARE_MODE_PWM2, or a prescaler overflow occurred.

tim_channel_t: Maps logical channel names to their hardware register offsets.

Enumerator	Value	Hardware register
TIM_CHANNEL_1	1	Maps to CCR1, CCMR1 bits [6:4], and CCER bit 0.
TIM_CHANNEL_2	2	Maps to CCR2, CCMR1 bits [14:12], and CCER bit 4.
TIM_CHANNEL_3	3	Maps to CCR3, CCMR2 bits [6:4], and CCER bit 8.
TIM_CHANNEL_4	4	Maps to CCR4, CCMR2 bits [14:12], and CCER bit 12.

tim_compare_mode_t: Output compare modes written to the OCxM field in CCMRx.

Enumerator	Value	Behavior
TIM_COMPARE_MODE_FROZEN	0x00	Output unchanged on compare match.
TIM_COMPARE_MODE_ACTIVE	0x01	Output set HIGH on compare match.
TIM_COMPARE_MODE_INACTIVE	0x02	Output set LOW on compare match.
TIM_COMPARE_MODE_TOGGLE	0x03	Output toggles on compare match.
TIM_COMPARE_MODE_PWM1	0x06	PWM mode 1 — output HIGH while CNT < CCR, LOW otherwise.
TIM_COMPARE_MODE_PWM2	0x07	PWM mode 2 — output LOW while CNT < CCR, HIGH otherwise.

4.2. Error Codes

All public TIM driver functions return tim_status_t. The three defined codes are listed below.

Code	Value	Definition
TIM_OK	0	All parameter checks passed and the register operation completed successfully.
TIM_ERROR	1	Reserved for generic hardware-level errors; not currently returned by any function.

TIM_INVALID	2	Returned when any input fails validation: unrecognized timer pointer, ms == 0 or > 65535, freq == 0, channel outside 1–4, mode > TIM_COMPARE_MODE_PWM2, or the auto-scaling loop in tim_setTimerFreq exhausted the 16-bit PSC range.

4.3. Function Definitions

tim_init

Field	Name / Value	Description
Function Name	`tim_init`	
Syntax	`tim_status_t tim_init(void)`	
Parameters (in)	None	—
Parameters (inout)	None	—
Parameters (out)	None	—
Return value	tim_status_t	Always TIM_OK. No validation performed.
Description	Disables the peripheral clocks for all supported timers by AND-NOT writing APB2ENR (TIM1, TIM9, TIM10, TIM11) and APB1ENR (TIM2, TIM3, TIM4, TIM5). Places the entire TIM subsystem in a known disabled state before individual timers are configured.	
Constraints	Must be called once at startup before any tim_initTimer call if a clean reset state is required. Does not reset individual timer registers — use tim_initTimer for per-timer register reset.	
Available via	tim_driver.h	

tim_initTimer

Field	Name / Value	Description
Function Name	`tim_initTimer`	
Syntax	`tim_status_t tim_initTimer(TIM_TypeDef *tim)`	
Parameters (in)	tim	TIM_TypeDef* — CMSIS pointer to the target timer (TIM1 … TIM5, TIM9 … TIM11).
Parameters (inout)	None	—

Parameters (out)	None	—
Return value	tim_status_t	TIM_OK on success. TIM_INVALID if is_valid_tim() fails.
Description	Enables the clock for the specified timer by setting its APB1ENR or APB2ENR bit via an if-else chain keyed on the CMSIS pointer. Resets CR1, PSC, CNT, and SR to 0, and sets ARR to 0xFFFF as a safe maximum count. This leaves the timer in a stopped, cleanly initialized state ready for configuration.	
Constraints	Clock remains enabled until explicitly cleared. Does not configure PSC or ARR for any specific interval — call tim_setTimerMs or tim_setTimerFreq next.	
Available via	tim_driver.h	

tim_setTimerMs

Field	Name / Value	Description
Function Name	`tim_setTimerMs`	
Syntax	`tim_status_t tim_setTimerMs(TIM_TypeDef *tim, uint32_t ms)`	
Parameters (in)	tim	TIM_TypeDef* — pointer to the target timer.
Parameters (in)	ms	uint32_t — desired interval in milliseconds (1 … 65535).
Parameters (inout)	None	—
Parameters (out)	None	—
Return value	tim_status_t	TIM_OK on success. TIM_INVALID if is_valid_tim() fails or ms is 0 or > 65535.
Description	Configures the timer for a blocking delay. PSC is set to (TIM_SYSTEM_CLOCK_HZ / 1000) - 1 = 15999, yielding a 1 kHz tick rate where each tick equals one millisecond. ARR is set to ms - 1 so the update event fires exactly ms ticks later. CNT and SR are cleared to prevent residual state from a previous run from causing an immediate spurious event.	
Constraints	Assumes TIM_SYSTEM_CLOCK_HZ = 16 MHz. The timer clock must be enabled with tim_initTimer before calling. Valid only for 16-bit timers (ARR ≤ 65535).	
Available via	tim_driver.h	

tim_setTimerFreq

Field	Name / Value	Description
Function Name	`tim_setTimerFreq`	
Syntax	`tim_status_t tim_setTimerFreq(TIM_TypeDef *tim, uint32_t freq)`	
Parameters (in)	tim	TIM_TypeDef* — pointer to the target timer.
Parameters (in)	freq	uint32_t — desired update frequency in Hz. Must be > 0 and achievable within 16-bit PSC and ARR.
Parameters (inout)	None	—
Parameters (out)	None	—
Return value	tim_status_t	TIM_OK on success. TIM_INVALID if tim is unrecognized, freq is 0, or PSC would exceed 0xFFFF.
Description	Calculates period = TIM_SYSTEM_CLOCK_HZ / freq. If period exceeds 0xFFFF, PSC is incremented until period / (PSC + 1) fits in 16 bits. If PSC itself would exceed 0xFFFF, TIM_INVALID is returned. PSC, ARR, CNT, and SR are all written; CNT and SR are cleared to avoid stale state. Used by the PWM module to set the PWM carrier frequency.	
Constraints	Assumes 16 MHz system clock. Resolution decreases as freq decreases (larger PSC means coarser ARR steps). Valid ARR range is 0 … 65535.	
Available via	tim_driver.h	

tim_enableTimer

Field	Name / Value	Description
Function Name	`tim_enableTimer`	
Syntax	`tim_status_t tim_enableTimer(TIM_TypeDef *tim)`	
Parameters (in)	tim	TIM_TypeDef* — pointer to the target timer.
Parameters (inout)	None	—
Parameters (out)	None	—
Return value	tim_status_t	TIM_OK on success. TIM_INVALID if is_valid_tim() fails.

Description	Starts the timer counter by setting the CEN bit in CR1 (CR1 |= TIM_CR1_CEN). Once set, the counter begins incrementing on each tick. Called by timer_delay_ms after tim_setTimerMs, and by pwm_start after the PWM channel is configured.
Constraints	PSC and ARR should be configured before calling, otherwise the timer runs with default (possibly unintended) values. Port clock must be enabled.
Available via	tim_driver.h

tim_disableTimer

Field	Name / Value	Description
Function Name	`tim_disableTimer`	
Syntax	`tim_status_t tim_disableTimer(TIM_TypeDef *tim)`	
Parameters (in)	tim	TIM_TypeDef* — pointer to the target timer.
Parameters (inout)	None	—
Parameters (out)	None	—
Return value	tim_status_t	TIM_OK on success. TIM_INVALID if is_valid_tim() fails.
Description	Halts the timer counter by clearing the CEN bit in CR1 (CR1 &= ~TIM_CR1_CEN). Called by timer_delay_ms after tim_waitTimer returns, and by pwm_stop after the compare channel is disabled.	
Constraints	Clearing CEN freezes CNT at its current value. SR and CNT are not reset; call tim_setTimerMs or tim_setTimerFreq before the next run to ensure a clean start.	
Available via	tim_driver.h	

tim_waitTimer

Field	Name / Value	Description
Function Name	`tim_waitTimer`	
Syntax	`tim_status_t tim_waitTimer(TIM_TypeDef *tim)`	
Parameters (in)	tim	TIM_TypeDef* — pointer to the target timer.
Parameters (inout)	None	—

Parameters (out)	None	—
Return value	tim_status_t	TIM_OK once the UIF flag is detected and cleared. TIM_INVALID if is_valid_tim() fails.
Description	Spins in a busy-wait loop reading SR & TIM_SR_UIF until the bit is asserted, indicating that the counter has reached ARR and the update event fired. The flag is then cleared by writing SR &= ~TIM_SR_UIF. This is a fully blocking call. Used exclusively by timer_delay_ms to implement the delay.	
Constraints	This is a blocking polling loop — the CPU is fully occupied during the wait. Not suitable for use inside an ISR or time-critical concurrent task. The timer must have been started with tim_enableTimer before calling.	
Available via	tim_driver.h	

tim_setTimerCompareChannelValue

Field	Name / Value	Description
Function Name	`tim_setTimerCompareChannelValue`	
Syntax	`tim_status_t tim_setTimerCompareChannelValue(TIM_TypeDef *tim, tim_channel_t channel, uint32_t value)`	
Parameters (in)	tim	TIM_TypeDef* — pointer to the target timer.
Parameters (in)	channel	tim_channel_t — target channel (TIM_CHANNEL_1 … TIM_CHANNEL_4).
Parameters (in)	value	uint32_t — compare threshold (0 … ARR). For PWM, determines the duty cycle.
Parameters (inout)	None	—
Parameters (out)	None	—
Return value	tim_status_t	TIM_OK on success. TIM_INVALID if is_valid_tim() or is_valid_channel() fails.
Description	Writes value to the CCR register for the specified channel via a switch statement (CCR1 for channel 1, CCR2 for channel 2, etc.). In PWM mode 1, the output is HIGH while CNT < CCR and LOW otherwise, so CCR directly controls the duty cycle as a fraction of ARR + 1.	

Constraints	OCxPE (preload enable) should be set via tim_setTimerCompareMode before changing CCR at runtime, so the new value takes effect at the next update event rather than immediately.
Available via	tim_driver.h

tim_setTimerCompareMode

Field	Name / Value	Description
Function Name	`tim_setTimerCompareMode`	
Syntax	`tim_status_t tim_setTimerCompareMode(TIM_TypeDef *tim, tim_channel_t channel, tim_compare_mode_t mode)`	
Parameters (in)	tim	TIM_TypeDef* — pointer to the target timer.
Parameters (in)	channel	tim_channel_t — target channel.
Parameters (in)	mode	tim_compare_mode_t — desired mode (must be <= TIM_COMPARE_MODE_PWM2).
Parameters (inout)	None	—
Parameters (out)	None	—
Return value	tim_status_t	TIM_OK on success. TIM_INVALID if any argument fails validation.
Description	Selects CCMR1 (channels 1–2) or CCMR2 (channels 3–4). Clears the 3-bit OCxM field at position [6:4] for odd channels or [14:12] for even channels, then writes the new mode value. Also sets the OCxPE (output compare preload enable) bit in the same register write for glitch-free duty cycle updates. Sets ARPE in CR1 to buffer ARR changes. This function must be called before tim_enableTimerCompareChannel to ensure the channel is in the correct mode before its output is enabled.	
Constraints	mode must be a valid tim_compare_mode_t value (0x00, 0x01, 0x02, 0x03, 0x06, or 0x07). Values in the gap (0x04, 0x05) are not valid PWM or compare modes and will cause TIM_INVALID.	
Available via	tim_driver.h	

tim_enableTimerCompareChannel

Field	Name / Value	Description

Function Name	`tim_enableTimerCompareChannel`	
Syntax	`tim_status_t tim_enableTimerCompareChannel(TIM_TypeDef *tim, tim_channel_t channel)`	
Parameters (in)	tim	TIM_TypeDef* — pointer to the target timer.
Parameters (in)	channel	tim_channel_t — target channel.
Parameters (inout)	None	—
Parameters (out)	None	—
Return value	tim_status_t	TIM_OK on success. TIM_INVALID if is_valid_tim() or is_valid_channel() fails.
Description	Sets the CCxE bit in CCER for the specified channel. The bit position is (channel - 1) * 4, placing CC1E at bit 0, CC2E at bit 4, CC3E at bit 8, and CC4E at bit 12. Enabling the bit connects the output compare logic to the physical pin. Called by pwm_start after all PWM parameters are configured.	
Constraints	The compare mode should be set with tim_setTimerCompareMode before enabling the channel output. If enabled before the mode is configured the channel output may be in an undefined state.	
Available via	tim_driver.h	

tim_disableTimerCompareChannel

Field	Name / Value	Description
Function Name	`tim_disableTimerCompareChannel`	
Syntax	`tim_status_t tim_disableTimerCompareChannel(TIM_TypeDef *tim, tim_channel_t channel)`	
Parameters (in)	tim	TIM_TypeDef* — pointer to the target timer.
Parameters (in)	channel	tim_channel_t — target channel.
Parameters (inout)	None	—
Parameters (out)	None	—

Return value	tim_status_t	TIM_OK on success. TIM_INVALID if either argument fails validation.
Description	Clears the CCxE bit in CCER for the specified channel (CCER &= ~(1 << offset)), disconnecting the output compare logic from the physical pin and stopping the PWM output. Called first by pwm_stop before disabling the timer.	
Constraints	After disabling the channel the pin retains its last output state. The GPIO pin's alternate function configuration is not changed.	
Available via	tim_driver.h	

4.4. Internal Function Definitions

is_valid_tim

Field	Name / Value	Description
Function Name	`is_valid_tim`	
Syntax	`static int is_valid_tim(TIM_TypeDef *tim)`	
Parameters (in)	tim	TIM_TypeDef* — timer pointer to validate.
Parameters (inout)	None	—
Parameters (out)	None	—
Return value	int	1 if tim matches one of the eight known CMSIS timer pointers; 0 if NULL or unrecognized.
Description	Private helper that validates the timer pointer against the eight supported peripherals (TIM1–TIM5, TIM9–TIM11) using sequential equality comparisons. Called at the entry of every public function to prevent writes to unrecognized memory addresses.	
Constraints	Defined as static in tim_driver.c — not visible outside the driver.	

is_valid_channel

Field	Name / Value	Description
Function Name	`is_valid_channel`	

Syntax	`static int is_valid_channel(tim_channel_t channel)`	
Parameters (in)	channel	tim_channel_t — channel identifier to validate.
Parameters (inout)	None	—
Parameters (out)	None	—
Return value	int	1 if channel is within TIM_CHANNEL_1 … TIM_CHANNEL_4; 0 otherwise.
Description	Private helper that confirms the channel argument is within the valid range 1–4. Used by all compare-channel functions to prevent out-of-bounds CCR or CCER register access.	
Constraints	Defined as static in tim_driver.c — not visible outside the driver.	

5. Future Enhancements

The following improvements may be considered for future revisions of the TIM driver:

- **Interrupt support:** Enable update interrupts (UIE in DIER) and configure NVIC priority, replacing the busy-wait loop in tim_waitTimer with an ISR-based approach for non-blocking operation.
- **DMA support:** Enable timer DMA requests (UDE in DIER) to allow periodic memory transfers or waveform generation without CPU involvement.
- **Input capture mode:** Extend CCMRx configuration to support input capture channels for measuring external signal frequencies or pulse widths.
- **32-bit timer support:** TIM2 and TIM5 on the STM32F411RE have 32-bit counters. A separate API variant or a mode flag could allow ARR to exceed 0xFFFF for very low frequencies.
- **Encoder interface mode:** Configure a timer pair in encoder mode (SMS field in SMCR) for quadrature encoder reading in motor control applications.
- **Center-aligned PWM:** Support CMS bits in CR1 for center-aligned (phase-correct) PWM, which produces lower harmonic distortion than edge-aligned mode.
- **One-pulse mode:** Set OPM in CR1 to make the timer automatically stop after one update event, useful for generating precise single pulses.

- **Complementary output support:** For TIM1 (an advanced-control timer), enable the complementary output channel (CCxNE in CCER) and configure dead time (DTG in BDTR) for half-bridge and H-bridge motor drive.
- **PLL clock scaling:** Parametrize TIM_SYSTEM_CLOCK_HZ to accept a runtime value so the prescaler calculations remain correct when the PLL is enabled and the system runs at frequencies other than 16 MHz.